

Local Search Algorithms

Motivation

Why do we need Local Search?

Many problems require only the **final solution**, not the path.

Examples:

- Scheduling problems
- Traveling Salesman Problem
- Machine learning parameter tuning

Classical search algorithms like BFS, DFS, and A* require large memory because they build large search trees.

Local search focuses only on improving the **current solution**.

Example: Class Scheduling Problem

Goal: Assign courses to time slots and rooms.

State

A possible schedule of classes.

Example:

Course	Time	Room
CSE101	9:00	R201
CSE221	9:00	R201
CSE330	10:30	R105

Problem: Two courses cannot be in the same room at the same time.

Neighbor

Modify the schedule slightly.

Example:

- Move CSE221 from 9:00 to 10:30
- Swap two course time slots

Local search repeatedly improves the schedule by reducing conflicts.

Idea of Local Search

Local search improves a candidate solution iteratively.

Basic process:

- 1 Start with an **initial state**
- 2 Generate **neighbor states**
- 3 Move to a **better neighbor**
- 4 Repeat until no improvement

Key idea:

- No search tree
- No path storage
- Only the **current state** is maintained

Memory usage is therefore **very small**.

State and Neighbor

Two important concepts in Local Search:

State

A candidate solution to the problem.

Example (N-Queens representation):

Column: 1 2 3 4 5 6 7 8

Row: 4 2 7 3 6 8 5 1

Neighbor

A small modification of the current state.

Example:

- Move one queen to another row

Neighbors are used to explore better solutions.

Hill Climbing Algorithm

Hill climbing is the simplest local search algorithm.

Idea:

Always move to the **neighbor with the best value**.

Steps:

- 1 Start with an initial state
- 2 Generate neighbors
- 3 Choose the best neighbor
- 4 Move to that neighbor if it improves the value
- 5 Stop if no improvement exists

Analogy:

Climbing a hill to reach the highest point.

Hill Climbing Pseudocode

Hill Climbing

```
while True:  
    neighbor = best_neighbor(current)  
    if value(neighbor) <= value(current): return current  
    current = neighbor
```

Evaluation function measures how good a state is.

Problems with Hill Climbing

Hill climbing may fail due to several problems.

Local Maximum

- A state better than its neighbors
- But not the global optimum

Plateau

- Large flat region
- Neighboring states have equal values

Ridge

- Improvement requires multiple sideways moves

These situations prevent reaching the global optimum.

Improving Hill Climbing

Several strategies can improve hill climbing.

Random Restart Hill Climbing

- Run hill climbing from multiple random states

Stochastic Hill Climbing

- Randomly choose among improving neighbors

Simulated Annealing

Simulated annealing allows occasional moves to worse states.
Inspired by the cooling process in metallurgy.
Acceptance probability:

$$P = e^{-\Delta E/T}$$

Where:

- ΔE = change in cost
- T = temperature

High temperature $\rightarrow$ more exploration

Low temperature $\rightarrow$ stable convergence

Simulated Annealing Algorithm

Idea: Escape local maxima by occasionally allowing worse moves, while gradually reducing their frequency.

Algorithm

- ① $C = C_{init}$ (current state)
- ② For $T = T_{max}$ down to T_{min} (temperature decreases)
 - ① $E_C = E(C)$ (energy / value of current state)
 - ② $N = Next(C)$ (generate neighboring state)
 - ③ $E_N = E(N)$ (energy of neighbor)
 - ④ $\Delta E = E_N - E_C$ (energy difference)
 - ⑤ If $\Delta E > 0$
 $C = N$ (move to better state)
 - ⑥ Else if $e^{\Delta E/T} > rand(0,1)$
 $C = N$ (accept worse state probabilistically)
- ③ End

Key Idea

- High $T \rightarrow$ more exploration
- Low $T \rightarrow$ more stable search

Applications of Local Search

Local search algorithms are widely used in:

- Scheduling problems
- Machine learning optimization
- VLSI design
- Feature selection
- Bioinformatics problems
- Traveling Salesman Problem

They are effective for solving large optimization problems.

Key points:

- Local search focuses on improving a single solution
- Requires very little memory
- Hill climbing is simple but may get stuck
- Simulated annealing improves exploration
- Useful for large optimization problems

Thank You